CpSc 418

Homework 4

Architecture

Due: October 28, 2025, 11:59pm
Early-Bird: Octover 26, 2025, 11:59pm

Prelude

Please submit your solution using:
`handin cs-418 hw4` Your solution should contain at least:

`hw4.pdf`: PDF for your written answers.

`dekker.c`: Your modified `dekker.c`.

`hw4.erl`: a template is provided at hw4.erl().

More useful files for this assignment are avalable at `hw4.html`.

Please submit code that compiles without errors or warnings. If your code does not compile, we might give you zero points on all of the programming problems. If we fix your code to make it compile, we will take off lots of points for that service. If your code generates compiler warnings, we will take off points for that as well, but not as many as for code that doesn't compile successfully. Any question about whether or not your code compiled as submitted will be determined by trying it on CS department linux machines.

1 The Questions (140 points + 1 Extra Credit)

0. **Collaborators (2 points (+1 extra credit)).**
In your `hw4.pdf` list anyone you collaborated with or outside materials that you used.

   - If the collaborator is a student in this class, give their CWL.
   - If the colloborator is a human who is not in this class, give their name and a short (i.e. one sentence) description of their relevance to this course.
   - If you used on-line material, give the URL.
   - If you used an AI assistant, give the name of the assistant. We'll give extra credit if you include your prompt, and one sentence to say what was useful in the response.
   - Other sources such as books should be just cited clearly.

   For each collaborator, list what questions you collaborated on.
   Anthropic Sonnet 4.5 for some questions, latex formatting, and testing.

   Contributions:
   1.b Latex table generation based on data.
   1.c Helped with this question, conceptual
   2.d I asked it to format the table based on erlang results and calculate max congestion formula.
   2.e Partial code generation, listed in hw4.erl
   2.f Understood how to solve the problem, had sonnet implement solution
   2.g Generated the proof based on 2.f observations
   3.a Generated the table based on erlang results, calculated formula
   3.b Generated the table based on erlang results, gave conclusions
   3.c Generated par_len based on a description and previous reduce patterns from `mt1.erl`
   3.d Generated par_len_ms based on sequential pattern
   3.e Generated a testing suite (hw4_test) based on finding primes from hw1, multiple worker count

testing
4.a-e Helped with these problems, conceptual

You do not need to list course materials including anything on the cs-418 website, assigned readings, the official Erlang documentation, or material from office hours or tutorials.

If you had no collaborators, write "*No collaborators*". A blank or omitted response will get no credit.

Extra Credit: When grading HW 1, one of our TA's pointed out that many of you did an extra fantastic job on this question providing detailed prompts that you used with AI assistants. We really appreciate this as we are learning how we can best leverage emerging technologies in our teaching. In fact, we appreciated it so much that we gave a point of extra credit for answers that the TA flagged as extra good! Thanks.

Going forward, we want to make this opportunity to everyone, whether or not you use AI. Here are *five* ways you can get that extra credit point:

(a) Include your prompts to AI assistants as described above.

(b) Tell us why you didn't use an AI assistant or didn't find it helpful.

(c) Write a few sentences that you think would be the best advice you could give to a student starting this assignment based on your experience of completing it.

(d) Identify a question that you thought was particularly helpful or pointlessly tedious, and give us a few sentences saying why.

(e) Write something else that you think is worthy of extra-credit. Mark it clearly as your response to the **Extra Credit** part of this question.

1. **Locks and Memory Coherence.** (30 points)
The source code for this assignment includes:

- `cx.c`: an implementation of a compare-and-exchange lock.
- `dekker.c`: an implementation of Dekker's mutual exclusion algorithm.
- `lock.c`: a test fixture for locks.

- `util.c`: a few functions used in Questions 1 and 2.
- `lock.h`: a header file shared by `cx.c`, `dekker.c`, `lock.c`, and `util.c`.
- `array.c`: functions for creating and simple computations on arrays of `int`s.
- `op.c`: test fixture for measuring execution times for the `ar_op` function from `array.c`.
- `top.h`: a header file shared by `array.c`, `op.c` and `util.c`.
- `Makefile`: a Makefile

The implementation of Dekker's mutual exclusion algorithm is based on the version presented in class. I've divided it up into separate functions for creating, destroying, acquiring, and releasing locks. As noted in lecture, Dekker's algorithm fails on a modern CPU where the cache coherency protocol does not guarantee sequential consistency. If you build the `lock` target and try the command:

```
lock dekker 10000
```

it will print out the number of errors that it encounters. The parameters to the `lock` program are:

```
lock which-lock ntrials
```

where

`which-lock` is `dekker` for Dekker's algorithm, and `cx` for the compare-and-exchange lock described below.

`ntrials` is the number of trials to run. Each thread acquires (and later releases) the lock `ntrials` times. During each trial, it counts to 10, setting the global variable `count[thread_id]` to the loop index with each iteration of the loop, and checking that `count[!thread_id]` is zero, i.e. that the other thread is not in it's critical region as well.

The first two questions:

(a) **(5 points)**: Identify the cause of the errors. In other words, explain how weak-consistency makes it possible for Dekker's algorithm to fail. Be specific. You should be able to give a specific sequence of operations in the two threads that leads to an error. This explanation should include the specific load, store, and branch operations that led to the error. Describe the specific memory operations and what happens in the cache and memory. You may assume that the cache uses a write-buffer.

Explain (briefly, not a lot of technical detail required) how weak-memory consistency led to the error. In particular, what operation(s) in the explanation that you provided for the failure was (were) allowed by weak consistency but would not be allowed with sequential consistency. This question is basically asking you to summarize what was presented in class.

This is due to modern processers not guaranteeing sequential consistency. Writes are placed in a write buffers, and other processors may read the old lock flags allowing entry into critical regions.

(b) **(10 points)**: Here's a "fix" that is very much *not* recommended in real life. The error happens because some memory operations can occur in a different order than expected according to sequential consistency. We can "help" the memory system get the ordering "right" by forcing some delays in program execution at critical points. The function `twiddle(int n)` in `util.c` calls the `sin` function `n` times, thus wasting a bit of CPU time. Yay!

Add a call to `twiddle(n)` to the code for Dekker's algorithm to reduce the error rate. Explain why you inserted the call where you did. Report the error rate you observe for a few (e.g. five) values of `n`. How big does `n` need to be to observe no errors when `ntrials==1000000`? Exact values aren't needed. For example, if you double `n` until you get a value for which you see no errors when `ntrials==1000000`, that's good enough.

I placed twiddle at dekker.c:43 so that the thread backs off after writing false to its own flag. This prevents the thread from immediately trying to acquire the lock again and allows more time for the false write to propegate to memory.

Error Rate Observations:

Testing Dekker's algorithm with different numbers of n for twiddle(n) (starting at 160 and doubling):

n	Errors Detected
160	14
320	6
640	115
2560	45
5120	2
10240	0

At n = 10240, no errors were detected.

Calling `twiddle` is not a very satisfactory solution. We never know that we've waited long enough, and in most cases, we'll wait longer than needed. We could use a memory fence instead of calling `twiddle`, but we know that fences slow all of the processors down. Instead, we'll examine a solution using atomic compare-and-exchange, `cmpxchg`, instructions. For example,

`cmpxchg [%dst] %exp %val`
reads the value at memory location whose address is given by register `%dst`. If this value matches the value in register `%exp` (the expected value), then the value of register `%val` is written to memory location `[%dst]`. If the values don't match, then the memory value is unchanged, but register `%exp` is updated to the value in memory.

The compare-and-exchange operation is performed as an *atomic* operation: no other reads or writes of memory location `[$dst]` can take place between the read and write. I'm most familiar with how this was done on a SPARC where the caches had a "read-with-intention-to-write" operation. The value at the specified address is read, and the cache moves to state `M` (or stays in `M` if already there). Any other caches holding the line for that address move to state `I`. The cache for the processor performing the compare-and-exchange operation will hold on to the line (even if there are pending reads or writes from other processors), until the local processor signals that the instruction has completed. This makes it essential that the read, compare, and write are done as a single instruction – the CPU cannot be interrupted in the middle of this sequence.

It is simple to implement a lock using compare-and-exchange. We assign a unique, non-zero, `int` valued id to each thread. The lock is implemented with a shared, `int` valued memory location, `lock->data`.

- When `lock->data == 0`, the lock is free.
- When `lock->data != 0`, `lock->data` is the thread-id of the thread holding the lock.
- To acquire the lock, a thread spins, reading `lock->data` until it reads a value of `0`.
  - The thread then does a compare-and-exchange operation to set `lock->data` to its thread-id if `lock->data` is still 0.
  - If the compare-and-exchange fails, then the thread goes back to spinning until `lock->data==0`.
- To release the lock, the thread that owns the lock sets `lock->data` to `0`.

Unlike Dekker's algorithm that can only provide mutual-exclusion between two threads, the compare-and-exchange lock can provide mutual-exchange between an arbitrary number of threads. A compare-and-exchange operation can operate on a single "word" which can be 4 or even 8 bytes. By using such a word to hold the thread-id of the lock owner, we can construct a lock that can handle a *very* large number of threads.

In this homework, we used `__atomic_...` primitives:

`__atomic_load_n(`*`type`*` *ptr, int memorder)`: Load a value from memory and return it.

> *`type`*: is the type of the value. *`type`* must be a type that the compiler recognizes as valid for atomic values. `int` is just fine.
>
> `ptr`: The memory location to be read.
>
> `memorder`: The compiler generates code so that the specified memory ordering holds for this operation. I was cautious and used `__ATOMIC_SEQ_CST`.

`__atomic_store_n(`*`type`*` *ptr, `*`type`*` val, int memorder)`: Store a value in memory.

> *`type`*: is the type of the value.
>
> `ptr`: The memory location to be updated.
>
> `val`: The value to write to `*ptr`.
>
> `memorder`: The compiler generates code so that the specified memory ordering holds for this operation. Again, I was cautious and used `__ATOMIC_SEQ_CST`.

`__atomic_compare_exchange_n(`*`type`*` *ptr, `*`type`*` *expected, `*`type`*` desired, bool weak, int success_memorder, int failure_memorder)`:
A compare-and-exchange operation. `gcc` (or `clang`, etc.) compiles this to a single machine instruction.

type: the type of the values in the operation.

`ptr`: The memory location to be tested, and updated if it presently holds the `*expected` value.

`expected`: `*expected` is value that we hope to find at `*ptr`. `expected` is a pointer because it will be updated with the value at `*ptr` if `*ptr != *expected`.

`desired`: The value to be written to memory at `ptr` if `*ptr == *expected`.

`weak`: Allow versions that "may fail spuriously", and the docs say "When in doubt, use `weak == false`. That's what I did.

`success_memorder`: Memory model for the compiler to ensure when `*ptr == *expected`.

`failure_memorder`: Memory model for the compiler to ensure when `*ptr != *expected`.

return value: `true` if the compare succeeds (i.e. `*ptr == *expected`), and `false` otherwise.

For more details, see gcc docs for built-in atomic operations.

With that explanation, we can now present our lock; see `cx.c`. The acquire function is (with a few assertions elided for clarity):

```
static void cx_acquire(Lock *lock, int thread_id) {
  int zero;
  int id1 = thread_id + 1;
  int *data = (int *)(lock->data);
  do {
    zero = 0;
    while(__atomic_load_n(data, __ATOMIC_SEQ_CST));
  } while (!__atomic_compare_exchange_n(data, &zero, id1, false, __ATOMIC_SEQ_CST,
                                                                 __ATOMIC_SEQ_CST));
}
```

And here are some comments:

`zero = 0`: `zero` is the value that we want to see in `*data` to be able to acquire the lock. Because `zero` is updated by `__atomic_compare_exchange_n` when `*data != zero`, we restore it for each iteration of the `do { ... }` loop.

`while(__atomic_load_n(data, __ATOMIC_SEQ_CST))`: spin until the lock is free. The x86 assembly generated by `gcc -O2 -S cx.c` for this loop is:

```
.L2:  movl  (%rdx), %eax     ; %eax <- *data
      testl %eax, %eax       ; set condition flags
      jne   .L2              ; continue if *data != 0
```

The read of `*data` is done with a single instruction, and no special treatment for the memory model.

`} while (!__atomic_compare_exchange_n(...))`: The atomic compare-and-exchange. If it fails, we go back to spinning until `*data == 0`. If it succeeds, we've acquired the lock and continue. The x86 assembly generated by `gcc -O2 -S cx.c` for this is:

```
lock cmpxchgl %esi, (%rdx)
jne .L2
ret
```

Looking back a few lines in the assembly code (you can give the command `gcc -O2 -S cx.c`), we can determine that `$esi` is `id1` and `%drx` is `data`. Register `%eax` is *always* used for the expected value – this is x86 assembly code; I don't explain its odd quirks.

An optimization that I noted is that there is no register for `zero`. That's when the

```
while(__atomic_load_n(data, __ATOMIC_SEQ_CST));
```

loop terminates, the value from the load, which is in register `%eax` must be `0`. The compiler uses this `0` for `expected`. Executing `cmpxchgl` sets the x86 condition flags. If the value of `*data` not `0`, the `jne` goes back to the `while`-loop. Otherwise, the lock is successfully acquired, and the function returns.

Releasing a lock is much simpler: just store `0` at `lock->data`. See `cx_release` in `cx.c`.

And finally, ...**More questions!**

(c) (15 points) Consider an execution with N threads numbered 1 through N. Assume that thread 1 holds the lock, and that the other threads are spinning. In this situation, threads 2 through N hold the cache line for `lock->data` in the `S` state. For simplicity, assume that thread 1 holds this cache line in the `S` state as well. Describe the memory accesses that are performed and the cache-line state transitions (e.g. from `S` to `I` or vice-versa) when thread 1 releases the lock, one of the other threads acquires the lock, and the other threads resume spinning.

Phase 1: Thread 1 releases the lock

Thread 1 executes `__atomic_store_n(data, 0)` to release:

- Thread 1 cache: $S \rightarrow M$ (needs exclusive access to write)
- Thread 1 sends BusInv to all other caches
- Threads 2-N caches: $S \rightarrow I$ (invalidated)
- Thread 1 writes 0 to cache line, propagates to memory

Phase 2: Spinning threads detect lock is free

Threads 2-N are spinning in `while(__atomic_load_n(data))`:

- All N-1 threads have cache line in I state, must fetch from memory
- All issue memory reads (approximately) simultaneously
- Thread 1 cache: $M \rightarrow S$ (or E, responds to reads)
- Threads 2-N caches: $I \rightarrow S$ (all read value 0)

This requires O(N) memory reads.

Phase 3: One thread acquires the lock

All N-1 threads see lock is free and attempt `cmpxchgl`. Say thread 2 wins arbitration:

- Thread 2 executes `cmpxchgl` (read-with-intent-to-write)
- Thread 2 cache: $S \rightarrow L$ (locked during atomic operation)
- Thread 2 sends BusInv to all other caches
- Thread 1, threads 3-N caches: $S \rightarrow I$ (invalidated)
- Thread 2 compares (expected=0, actual=0), writes thread_2_id
- Thread 2 cache: $L \rightarrow M$
- `cmpxchgl` succeeds

Phase 4: Other threads fail to acquire and resume spinning

Each of threads 3-N attempts `cmpxchgl`:

- Thread k (for $k \in \{3, ..., N\}$) executes `cmpxchgl`
- Thread k cache: $I \rightarrow L$ (locked during read-with-intent-to-write)
- Thread k must wait for thread 2's cache to finish (bus arbitration)
- Thread k reads value (thread_2_id), compares with expected (0), fails
- Thread k cache: $L \rightarrow E$ or M (has exclusive copy momentarily)

- Each failed `cmpxchgl` sends BusInv, invalidating other spinning threads
- Thread k returns to `while` loop, reads lock value again
- Thread k cache: likely I again after other threads' `cmpxchgl` attempts

After all failed attempts, threads resume spinning:

- Thread 2 cache: M (holds lock)
- Threads 3-N must read from thread 2's cache or memory
- Threads 3-N caches: $I \rightarrow S$ (read non-zero value)
- Thread 2 cache: $M \rightarrow S$ (shared with spinning threads)

Big-O Analysis:

The compare-and-exchange lock generates $\Theta(N^2)$ inter-cache and memory activity:

- Release: O(N) invalidations
- All threads read lock is free: O(N) memory reads
- Each of N-1 threads attempts `cmpxchgl`: each attempt causes O(N) invalidations
- Total: $O(N) + O(N) + O(N) \cdot O(N) = O(N^2)$ cache coherence traffic

This excessive traffic is why more sophisticated locking algorithms (e.g., queue-based locks, MCS locks) are preferred for high-contention scenarios.

Hints: there are many details I could add about exactly how a cache handles read-with-intent to write, and so on. Here are some assumptions that I'll suggest to keep the problem simple:

- When a processor does a "read-with-intent-to-write", if the cache line is in the `I` or `S` states, the value is read from memory, *and* the other caches are signaled with *BusInv* to invalidate their entries for that line. If the cache line for the address is already in states `E` or `M`, no action with external memory is needed. In either case, cache moves to a new state, `L`. When the processor writes to the address, the cache-line moves to state `M`. If the compare part of the compare-and-swap fails, then the processor signals to the cache to move to state `M`.[1] While the cache is in state `L`, it signals a stall to any other cache that requests the cache line through a memory read or write. The operation is delayed until the cache line moves to state `M` or `E`, and then the operation is handled according to the usual MESI protocol. There are many ways we could optimize this protocol, but I want to keep it simple.
- When threads 2 through N are spinning, you can assume that they all attempt to load from `lock->data` on the same clock cycle. If the loads hit in their caches, then they all get the data locally; otherwise, each cache misses and they each issue a memory read.
- When the thread for a cache starts a `cmpxchgl` instruction, all other caches invalidate their lines for `lock->data`. If multiple threads attempt a `cmpxchgl` at the same time, there is memory-bus arbitration in the hardware that makes the threads do these operations one at a time.

Hint: you should find that a compare-and-exchange lock generates a lot of inter-cache and memory activity when multiple threads are spinning and waiting for the lock. Give a big-$\mathcal{O}$ characterization of how much memory and inter-cache activity occurs. There are better algorithms for locking. By working this example, you should get a good sense of how protocols like MESI work in real life.

[1] Note: the cache moves to state `M` because it could have entered state `L` from any of the other states, i.e. `M`, `E`, `S`, or `I`. If the cache was in state `M` when the read-with-intent-to-write operation starts, it must move to state `L` to prevent any state transitions until the read-with-intent to write operation finishes. Because he cache may have entered state `L` from state `M`, then we can't assume that main memory has the up-to-date value for the location. Thus, the cache moves to state `L` when leaving state `M`.

Of course, cache designers can make variations on this. We could have two version of the `L` state to indicate whether or not the cache line is dirty. Or, we could always do a write back at the end of the read-with-intent-to-write, just writing the value we read if the compare-and-exchange fails. I'm sure there are many other variations, and they've probably all been studied by those who have dedicated their careers to cache design.

Finally, a word about weak consistency. Instructions such as `cmpxchgl` ensure that the write is completed in main memory before the instruction completes. This makes the write visible to all other processors. This also means that all previous writes by that processor will have propagated to main memory. It makes no guarantee about pending writes from other processors.

2. Hypercube Congestion: (**42 points**)
Consider a d dimensional hypercube. For this problem, we will assume that d is even. For this problem, we will use dimension routing, where messages are first routed according to the least significant bit of their destination. When writing addresses below as lists below, we will write them with the least-significant bit at the left and the most significant bit at the right, i.e. "little-endian" ordering. Sadly, Erlang uses 1-based indexing; so, lists:nth(1, Addr) returns the 0^{th} bit of Addr, and more generally, `lists:nth(I+1, Addr)` returns I^{th} bit. For example if a node has address `[0,1,0,1]`, the least-significant bit of the address is `0`, and the most-significant bit is `1`.

The route when $d = 4$ node `[0,1,0,1]` to node `[1,1,0,0]` using dimension routing is:

```
[0,1,0,1]  →  [1,1,0,1]  % route on address bit 0
           →  [1,1,0,1]  % route on address bit 1
           →  [1,1,0,1]  % route on address bit 2
           →  [1,1,0,0]  % route on address bit 3
```

(a) (**3 points**) Describe the route from node `[1,0,1,1]` to node `[0,1,1,0]` in the same style as the example above.

```
[1,0,1,1]  →  [0,0,1,1]  % route on address bit 0
           →  [0,1,1,1]  % route on address bit 1
           →  [0,1,1,1]  % route on address bit 2
           →  [0,1,1,0]  % route on address bit 3
```

We say that there is congestion if two messages want to traverse the same link at the same step. For example, consider what happens when the two messages below are sent at the same time:

Message 1: the processor at node `[0,1,0,1]` sends a message to node `[1,0,0,1]`;

Message 2: the processor at node `[1,1,0,1]` sends a message to node `[1,0,1,0]`.

Using dimension routing, we get:

Message 1			Message 2			% Remark
`[0,1,0,1]`	→	`[1,1,0,1]`	`[1,1,0,1]`	→	`[1,1,0,1]`	% route on address bit 0
`[1,1,0,1]`	→	`[1,0,0,1]`	`[1,1,0,1]`	→	`[1,0,0,1]`	% congestion on address bit 1

Congestion is undesirable because it means that one of the messages has to wait in a buffer while the other is sent across the congested link. If a routing pattern causes congestion, then the program won't get the maximum bandwidth of the hypercube network.

(b) (**3 points**) Give a pair of routes of the form `Src1` → `Dst1` and `Src2` → `Dst2` such that both routes can be performed at the same time using dimension routing on a 4-dimensional hypercube without congestion. Describe how the messages move at each step as in the example above.

Message 1: the processor at node `[1,1,1,1]` sends a message to node `[1,1,1,0]`;

Message 2: the processor at node `[0,0,0,0]` sends a message to node `[0,0,1,0]`.

Message 1			Message 2			% Remark
[1,1,1,1]	→	[1,1,1,1]	[0,0,0,0]	→	[0,0,0,0]	% route on address bit 0
[1,1,1,1]	→	[1,1,1,1]	[0,0,0,0]	→	[0,0,0,0]	% route on address bit 1
[1,1,1,1]	→	[1,1,1,1]	[0,0,0,0]	→	[0,0,1,0]	% route on address bit 2
[1,1,1,1]	→	[1,1,1,0]	[0,0,0,0]	→	[0,0,1,0]	% route on address bit 3

(c) (**3 points**) Give a pair of routes of the form `Src1`→`Dst1` and `Src2`→`Dst2` such that congestion occurs when both routes are performed at the same time using dimension routing on a 4-dimensional hypercube. Describe how the messages move at each step as in the example above. I gave such an example above. To get credit, you must give a *different* example.

Message 1: the processor at node `[1,1,0,1]` sends a message to node `[0,0,1,0]`;
Message 2: the processor at node `[0,1,0,1]` sends a message to node `[0,0,1,1]`.

Message 1			Message 2			% Remark
[1,1,0,1]	→	[0,1,0,1]	[0,1,0,1]	→	[0,1,0,1]	% route on address bit 0
[0,1,0,1]	→	[0,0,0,1]	[0,1,0,1]	→	[0,0,0,1]	% congestion on address bit 1

We want an example that causes severe congestion in the hypercube. To define congestion we will first define a "traffic pattern". A traffic pattern is a list of `{Src, Dst}` tuples, where `Src` and `Dst` are node addresses. For each element of the list, a message is to be sent from the `Src` node `Dst`. We will say that a traffic pattern is crossbar-routable in one step if there are no two tuples with the same `Src` node, and there are no two tuples with the same `Dst` node.

A traffic pattern that is crossbar-routable in one step may cause congestion on a hypercube using dimension routing. For example, on a 4-dimensional traffic pattern the traffic pattern:

```
[{[0,1,0,1], [1,0,0,1]}, {[1,1,0,1], [1,0,0,0]}]
```

is cross-bar routable in one step but causes congestion on a hypercube using dimension routing when both messages need to use the dimension-1 link from node `[1,1,0,1]` to node `[1,0,0,1]`. We can create a traffic pattern where the congestion grows as the square root of the number of processors in the hypercube.

For simplicity, let `D`, the dimension of the hypercube be even. We can divide a node address, `Addr` into to sublists, `Left` and `Right`, each of length `D div 2` such that `Addr = Left++Right`. For example, if `D=8` and `Addr=[0,1,1,0, 1,0,1,1]`, then `Left=[0,1,1,0]` and `Right=[1,0,1,1]`. We will say that the *transpose* of `Addr` is `Right++Left`. `hw4:traffic(D)` generates the traffic pattern where each node in a `D`-dimensional hypercube sends a message to its transpose. This pattern is crossbar routable in one step because transpose is one-to-one. As you are about to prove, it also creates bad congestion for a hypercube.

(d) (**3 points**) The function

`hw4_lib:congestion_0(D) -> {MaxCongestion, MaxCongestedLinks}`

where `MaxCongestion` is the number of messages conveyed by the most congested link of a `D`-dimensional hypercube for the `hw4:traffic(D)` traffic-pattern. `MaxCongestedLinks` is the number of links that convey `MaxCongestion` messages.

Report `MaxCongestion` and `MaxCongestedLinks` for `D=2`, `4`, `6`, `8`, and `10`. Show from the date you reported that for these examples `MaxCongestion` $= \frac{1}{2}2^{D/2}$ and `MaxCongestedLinks` $= 2 * 2^{D/2}$.
test Running `hw4_lib:congestion_0(D)` for various values of D:

D	Reported MaxCongestion	Formula for MaxCongestion $\frac{1}{2}2^{D/2}$	Reported MaxCongestedLinks	Formula for MaxCongestedLinks $2 \cdot 2^{D/2}$
2	1	$\frac{1}{2}2^1 = 1$	4	$2 \cdot 2^1 = 4$
4	2	$\frac{1}{2}2^2 = 2$	8	$2 \cdot 2^2 = 8$
6	4	$\frac{1}{2}2^3 = 4$	16	$2 \cdot 2^3 = 16$
8	8	$\frac{1}{2}2^4 = 8$	32	$2 \cdot 2^4 = 32$
10	16	$\frac{1}{2}2^5 = 16$	64	$2 \cdot 2^5 = 64$

As shown in the table, the reported values match the formulas exactly for all tested values of D.

(e) (**10 points**) `hw4_lib:congestion_0(D)` gets rather slow as `D` grows. The function `hw4_lib:congestion(D)` is much faster, but you need to implement `hw4:link_counts(SrcDstList)` for the faster version to work.
Implement `hw4:link_counts(SrcDstList)`.

- There are many hints in the comments in `hw4.erl`.
- Why do this? In many ways, this is a fairly straightforward problem. I'm including it as a "check-point". If you understand dimension routing, and how we created the `SrcDstList` in `hw4:traffic(D)`, then writing `hw4:link_counts(SrcDstList)` should be straightforward. If it's not, then this is a good chance to figure out what you need to understand before you try the last two parts of this problem.
- Testing your implementation of `hw4:link_counts(SrcDstList)` should be easy: just check that `hw4_lib:congestion(D) =:= hw4_lib:congestion_0(D)`.

Implemented hw4.erl:link_counts/1

(f) (**5 points**) Identify any one of the 32 links that has conveys the `MaxCongestion` messages for `hw4_lib:congestion(8)` (or, equivalantly, for `hw4_lib:congestion_0(D)`). Describe the link in the form `{NodeAddress, K}` where `K` is the dimension associated with the link, i.e. `0 =< K < D`. State the eight `{Src,Dst}` pairs that created these eight messages for this link.

- You can solve this as a coding problem.
- Why do this? How you have a concrete example for the final part of this problem.
- If one concrete example isn't enough doesn't help you see how to solve the final part, then you might want to try generating more examples for different values of `D` or different maximally congested links.

Using the helper functions in `hw4.erl`:

`hw4:find_max_congested_links(8)` returns a map of all maximally congested links. Selecting one example link: `{[0,0,0,0,0,0,0,0],4}`.

The eight `{Src, Dst}` pairs that use this link are found with
`hw4:messages_using_link({[0,0,0,0,0,0,0,0],4}, 8)`.
// OUTPUT:

```
[{[1,0,0,0,0,0,0,0],[0,0,0,0,1,0,0,0]},
{[1,0,0,1,0,0,0,0],[0,0,0,0,1,0,0,1]},
{[1,0,1,0,0,0,0,0],[0,0,0,0,1,0,1,0]},
{[1,0,1,1,0,0,0,0],[0,0,0,0,1,0,1,1]},
{[1,1,0,0,0,0,0,0],[0,0,0,0,1,1,0,0]},
{[1,1,0,1,0,0,0,0],[0,0,0,0,1,1,0,1]},
{[1,1,1,0,0,0,0,0],[0,0,0,0,1,1,1,0]},
{[1,1,1,1,0,0,0,0],[0,0,0,0,1,1,1,1]}]
```

(g) (**15 points**) Show that for a `D` dimensional hypercube there is at least one link that conveys $\frac{1}{2}2^{D/2}$ messages when routing the `hw4:traffic(D)` pattern. This should be a mathematical argument that works for any `D` that is a positive, even integer. In other words, you should take your observations from the previous parts of this problem and use them to formulate and justify a general result.

Proof:

Consider the transpose traffic pattern on a D-dimensional hypercube where D is even. Each node with address `Addr` = `Left` + +`Right` (where |`Left`| = |`Right`| = $D/2$) sends a message to its transpose `Right` + +`Left`.

Key Observation: Consider the link from node $[0, 0, \ldots, 0]$ (all zeros) along dimension $D/2$.

Which messages use this link?

A message from `Src` to `Dst` uses this link if and only if:

i. After routing along dimensions 0 through $D/2 - 1$, the message reaches node $[0, 0, \ldots, 0]$

ii. The message needs to change bit $D/2$ (i.e., `Src`$[D/2] \neq$ `Dst`$[D/2]$)

Pattern of sources:

For the transpose pattern, consider sources of the form:

$$\texttt{Src} = [1, b_1, b_2, \ldots, b_{D/2-1}, 0, 0, \ldots, 0]$$

where the first $D/2$ bits can vary (with first bit fixed to 1), and the last $D/2$ bits are all 0.

The destination is:

$$\texttt{Dst} = [0, 0, \ldots, 0, 1, b_1, b_2, \ldots, b_{D/2-1}]$$

Routing analysis:

- Dimensions 0 through $D/2 - 1$: The message routes from `Src` to match the first $D/2$ bits of `Dst`, arriving at node $[0, 0, \ldots, 0, 0, \ldots, 0]$.
- Dimension $D/2$: The message must change from 0 to 1, using the link from $[0, 0, \ldots, 0]$ along dimension $D/2$.

Counting messages:

How many such source nodes exist? The pattern $[1, b_1, \ldots, b_{D/2-1}, 0, \ldots, 0]$ has:

- First bit: fixed to 1 (to ensure dimension $D/2$ link is used)
- Next $D/2 - 1$ bits: can be anything ($2^{D/2-1}$ choices)
- Last $D/2$ bits: all 0

Total messages using this link: $2^{D/2-1} = \frac{1}{2} \cdot 2^{D/2}$.

Conclusion: The link from node $[0, 0, \ldots, 0]$ along dimension $D/2$ conveys exactly $\frac{1}{2}2^{D/2}$ messages.

3. Queues: (**36 points**)

The Dragonfly paper frequently refered to the message queues in the Dragonfly router. This question is an introduction to simple queues so we can explore the impact of queues in message routing in a future assignment.

A common example for introducing queues is a retail store. Customers arrive at the store and subsequently at the cashier at a rate of λ customers per hour. We assume that the customer arrivals are statistically independent events – e.g. we don't have a bus or train that delivers a huge number of customers all at once and then a big gap until the next bus or train arrives. With this assumption of independence, it can be show that the amount of time between two customers arriving is exponentially distributed:

$$P\{(\text{time between customer arrivals}) > t\} \quad = \quad e^{-\lambda t}$$

We'll write m_a to denote the average time between the arrivals of two consecutive customers; $m_a = 1/\lambda$. The function `hw4_lib:expDist(M_a)` returns a function that provides samples from an exponential distribution with mean `M_a`. For example:

```
1> Exp2 = hw2_lib:expDist(2),
#Fun<hw4_lib.0.93202345>
2> [ Exp2() || _ <- lists:seq(1,10)].
[0.1713282082900304,1.6155340550767132,1.334425942758621,
 0.7614403269496659,3.307862051970613,0.44543298194790326,
 3.1505387643300136,0.6458216282019269,2.8517732640277695,
 5.386802172521248]
```

For simplicity we assume that the store has only one cashier and that the time for the cashier to check out a customer is also exponentially distributed with a mean of m_s. We write μ to denote the average *rate* of checking out customers; $\mu = 1/m_s$. A queue where the arrival times and service times are exponentially distributed and there is one server (e.g. the cashier) is calle a called a M/M/1 queue. M/M/1 queues have a mathematical simplicity that makes them the starting model for queues.

If a customer arrives at checkout while the cashier is busy, they form a queue. The length of the queue is the number of waiting customers, including any customer who is currently being serviced. This question explores the average length of this queue. For the M/M/1 queue the answer is known: If $m_a > m_s$, then the average queue length is $1/((m_a/m_s) - 1)$; otherwise, the queue grows without bound. We will show this by simulation instead of derivation, which has the advantage that we can use the same approach to variations on this queue where analytical approaches are impractical.

The function

`hw4_lib:sim_len_ms(N_trials, N_warmup, N_run, R_a, R_s)`

returns a keylist of the form

`[{mean, Mean}, {std, Std}]`

where `Mean` is the average queue length from the simulation and `Std` is the standard deviation. I'll describe the parameters right-to-left:

- `R_s` should be a function that takes no arguments. `R_s`() returns samples of the service time. For example if `R_s = hw4_lib:expDist(Mean)`, R_s() will return samples of an exponentially distributed random variable with mean `Mean`.
- `R_a` should be a function that takes no arguments that returns samples of the time between arrivals.
- `N_run`: the `Mean` and `Std` returned by `sim_len_ms` are computed over a sequence of `N_run` events where an event is either an arrival or a service.
- `N_warmup`: we start with an empty queue. Obviously, this is below the average queue length, and the expected queue length will asymptotically approach the limit value of $\left(\frac{m_a}{m_a} - 1\right)^{-1}$ as more events are processed. Thus, `sim_len_ms` simulates the queue for `N_warmup` steps *before* it starts to accumulate samples for computing the mean. I found that the results seem fairly stable if `N_warmup >= 10*MeanLen`^{2}, where `MeanLen` is the average queue length. Of course, this means you need to guess the queue length to measure it. I'll address this chicken-and-egg problem below.
- `N_trials`: `sim_len_ms` runs `N_trials` trials, and reports the mean and standard deviations frome them.

Now for the questions.

(a) **Measure the queue length using simulation.** (6 points)
`hw4_lib:sim_len_ms(100, 100000, 10000, hw4_lib:expDist(M_a), hw4_lib:expDist(1.0))` performs 100 trials, where each trial has a warm-up of 100,000 steps, and then computes the average queue length over 10,000 more steps. Measure the average queue length for `M_a` $\in$

$\{3, 2, 1.5, 1.1, 1.05\}$. Report your raw data using the median of 5 runs for each value of `M_a`. Provide a table with a row for `M_a` and a row for the corresponding average queue lengths (medians). Do the measurements seem reasonably close to the formula? Feel free to include a plot in your solution if you think it adds to the clarity.

Queue Length Simulation Results:

M_a	3.0	2.0	1.5	1.1	1.05
Median (Simulated)	0.4988	1.0013	2.0011	9.8975	21.1616
Formula: $1/(M_a - 1)$	0.5	1.0	2.0	10.0	20.0

Analysis: The measurements are very close to the formula values. The simulated medians closely match the theoretical formula $\frac{1}{M_a - 1}$ for all tested values:

- For $M_a = 3.0$: simulated 0.4988 vs. formula 0.5 (0.24% difference)
- For $M_a = 2.0$: simulated 1.0013 vs. formula 1.0 (0.13% difference)
- For $M_a = 1.5$: simulated 2.0011 vs. formula 2.0 (0.06% difference)
- For $M_a = 1.1$: simulated 9.8975 vs. formula 10.0 (1.03% difference)
- For $M_a = 1.05$: simulated 21.1616 vs. formula 20.0 (5.81% difference)

The agreement is excellent, especially for higher values of M_a. The slightly larger relative difference for $M_a = 1.05$ is expected as the queue becomes more sensitive to small fluctuations when operating closer to saturation ($M_a \approx 1$).

(b) **Impact of service time distribution.** (6 points)
Now consider

```
hw4_lib:sim_len_ms(100, 100000, 10000, hw4_lib:expDist(1.0), R_s)
```

for `R_s` $\in$ {`hw4_lib:expDist(1.0)`, `hw4_lib:constDist(1.0)`, `hw4_lib:erlangDist(1.0, 5)`}. Report your measurements. What patterns or trends do you observe? Feel free to try changing the distribution or mean for `R_a` or other experiments if this helps you understand the queue better.

Answers will be graded by your showing that you thought about the data that you saw, and tried experiments to test your hypotheses. You are welcome to find references about queues – make sure you cite them.

Service Time Distribution Impact Results:

With arrival rate fixed at $R_a = \text{expDist}(1.0)$, tested three different service time distributions:

Service Distribution R_s	**Median Queue Length**	**Std Dev**
expDist(1.0) (M/M/1)	260.81	185-210
constDist(1.0) (M/D/1)	187.20	137-149
erlangDist(1.0, 5) ($M/E_5/1$)	189.58	121-161

Analysis and Observations:

- **Impact of Service Variability:** The M/M/1 queue (exponential service) has the highest median queue length (260.81), while the M/D/1 queue (constant service) has the lowest (187.20). This demonstrates that reducing service time variability significantly reduces queue length.
- **Erlang Distribution:** The Erlang distribution with shape parameter 5 produces results (189.58) very close to the constant service time. This makes sense as Erlang(k) approaches a constant distribution as k increases - with $k = 5$ already showing much less variability than exponential.
- **Variance Effect:** All three distributions have the same mean service time (1.0), but different variances. Exponential has the highest variance (CV=1), constant has zero variance (CV=0), and Erlang(5) has intermediate variance (CV=$1/\sqrt{5} \approx 0.45$). Lower variance leads to shorter queues.

- **Standard Deviations:** The standard deviations are very large relative to the means (50-80% of mean), indicating high variability in queue lengths even within a single configuration, which is characteristic of queues operating near saturation ($\rho = 1$).
- **Practical Implications:** This shows that even when average service times are equal, predictability in service time significantly improves queue performance - a reduction of about 28% in queue length from M/M/1 to M/D/1.

(c) **Make it parallel: `par_len`.** (10 points)
Write `hw4:par_len/5` which is a parallel version of hw4_lib:`sim_len/4`. and hw4_lib:`sim_len_ms/4`. The function hw4_lib:`sim_len/4` returns a tuple structure for computing simple statistics. Use `wtree:reduce`. The function `stat:combine/2` can make your life easier from the course library – see the docs.

implemented in `hw4.erl` lines 219-230

(d) `par_len_ms` (4 points)
The function hw4_lib:`sim_len_ms/4` calls hw4_lib:`sim_len/4` and then uses the functions `stat:mean/1` and `stat:std/1` from the course library to compute the mean and standard deviation. You are encouraged to look at the code in hw4_lib`.erl` and use that as a template for your `par_len_ms` (implemented in `hw4.erl`).

implemented in `hw4.erl` lines lines 232-237

(e) **Measure Speed-Up** (10 points)
Measure the speed-up for your implementation of `par_len_ms`. Make your timing measurements on `thetis` or `gambier`. This time, you decide how to make your measurments. Please describe how you measured the speed-ups, and give your reasons for your design choices.

Methodology:

The performance benchmarking methodology for `par_len` closely follows the approach used in HW1 for parallel prime finding. The benchmark implementation can be found in `hw4_test.erl` (lines 262-365). The implementation uses the `time_it:t/1` function from the course library to measure execution times. Key similarities to HW1 methodology:

- **Baseline measurement**: Sequential version (`hw4_lib:sim_len_ms`) establishes baseline performance
- **Multiple worker configurations**: Tests with varying numbers of workers to identify optimal parallelism
- **Large workload**: Uses N_trials = 200, N_warmup = 10000, N_run = 5000
- **Timing methodology**: `time_it:t/1` runs the timed function repeatedly until at least 1 second elapsed
- **Speedup calculation**: Speedup = $\frac{\text{Sequential Time}}{\text{Parallel Time}}$

Results:

Measured on thetis.students.cs.ubc.ca with arrival rate R_a = expDist(1.5) and service rate R_s = expDist(1.0).

Workers	Time (s)	Speedup
1 (seq)	1.859	1.00×
2	0.947	1.96×
4	0.549	3.38×
8	0.254	7.33×
16	0.151	12.34×
32	0.116	16.01×
64	0.073	25.63×
128	0.060	30.85×
256	0.055	34.07×
512	0.055	33.78×
1024	0.055	33.60×

The implementation achieves approximately **34× speedup** at 256 workers.

Hint: This problem is a simple example of reduce. My `par_len` function is eight lines long, and my `par_len_ms` function is three lines. The purpose of this problem is to reinforce using reduce and scan with another example. It should have been on HW2, but I hadn't worked out a "gentle" introduction to queueing theory in time for that assignment. We are deliberately giving you more flexiblity for the code writing and measurements. This is to give you experience applying the ideas you've learned in class.

4. Work-Span and Dynamic Programming: **(30 points)**
Dynamic programming is a method for solving optimization problems when we have a way of breaking the problem into smaller problems, we can define an ordering of the sub-problems, and it's "easy" to solve a sub-problem once all of the sub-problems that are earlier in the ordering have been solved. A classical example is the editing distance between two strings: if we have two strings, s_1 and s_2, and have a costs associated with inserting a character, c_{ins}, a cost for deleting a character, c_{del}, and a cost for replacing a character with another one, c_{rpl}, what is the minimum cost sequence of operations that transforms string s_1 into string s_2? See, for example,
A general method applicable to the search for similarities in the amino acid sequence of two proteins, S.B. Needleman & C.D. Wunsch.

Let's say that s_1 has n characters and s_2 has m characters. We can solve the editing distance problem by filling in a $(n+1) \times (m+1)$ tableau, T. Initialize $T(0, 0:m)$ and $T(0:n, 0)$ with

$$T(0, j) = j * c_{ins}$$
$$T(i, 0) = i * c_{del}$$

For each (i, j) with $0 < i < n$ and $0 < j < m$, if we have filled in $T(i-1, j-1)$, $T(i-1, j)$, and $T(i, j-1)$, the we can set $T(i, j)$ according to:

$$T(i,j) = \begin{cases} \min(T(i-1,j-1), T(i-1,j) + c_{del}, T(i,j-1) + c_{ins}), & \text{if } s_1(i) = s_2(j) \\ \min(T(i-1,j-1) + c_{rpl}, T(i-1,j) + c_{del}, T(i,j-1) + c_{ins}), & \text{if } s_1(i) \neq s_2(j) \end{cases}$$

In English, this means:

- If we have already matched the first $i-1$ characters of s_1 to the first $j-1$ characters of s_2 with a cost of $T(i-1, j-1)$, then
  - if the i^{th} character of s_1 matches the j^{th} character of s_2, then we can match the first i characters of s_1 to the first j characters of s_2 with a cost of $T(i-1, j-1)$. In other words, there is no cost if the corresponding character match.

  - otherwise, we can match the first i characters of s_1 to the first j characters of s_2 with a cost of $T(i-1, j-1) + c_{rpl}$. In other words, we replace the i^{th} character of s_1 with the j^{th} character of s_2.

- If we have already matched the first $i-1$ characters of s_1 to the first j characters of s_2 with a cost of $T(i-1, j)$, then we can match the first i characters of s_1 to the first j characters of s_2 with a cost of $T(i-1, j) + c_{del}$, by deleting the i^{th} character of s_1.
- If we have already matched the first i characters of s_1 to the first $j-1$ characters of s_2 with a cost of $T(i, j-1)$, then we can match the first i characters of s_1 to the first j characters of s_2 with a cost of $T(i, j-1) + c_{ins}$, by inserting the j^{th} character of s_2 into s_1.

We choose the cheapest of these options for the cost of $T(i, j)$. Note that there is never an advantage of choosing a more expensive solution for $T(i, j)$ when finding solutions that are later in the tableau.

Of course, we want a parallel solution to this problem. There are many published papers on parallel implementations of dynamic programming algorithms. Here, we will examine the problem from the perspective of the work-span model. For the questions below, assume that s_1 and s_2 are both of the same length, n.

(a) (**5 points**)
What is the *Work* in the work-span model for solving the editing distance using the dynamic programming approach described above when s_1 and s_2 when are both of length n? Ignore the cost of initialization and just determine the cost of filling in $T(i, j)$ for $1 \leq i \leq n$, and $1 \leq j \leq n$. If you want a working example, try filling in the tableau where s_1 is `"length"` and s_2 is `"slings"`.
$T_1 = \Theta(n^2)$. We fill an $n \times n$ tableau, with each cell $T(i, j)$ requiring $O(1)$ work to compute the minimum of three values.

(b) (**5 points**)
What is the *Span* in the work-span model when s_1 and s_2 are both of length n?
$T_\infty = \Theta(2n)$. Using diagonal parallelism, cells where $i + j = k$ can be computed in parallel. The diagonals range from $k = 2$ to $k = 2n$, requiring $2n - 1 \approx 2n$ sequential steps.

(c) (**5 points**)
Let $SpeedUp(n, p)$ denote the *optimal* speed-up according to work-span when using p processors and s_1 and s_2 are of length. What is the lower bound from $SpeedUp(n, p)$ given by Brent's lemma. You may start by stating the definition of Brent's lemma using T_1, T_∞, and p, but that's not a complete answer. Use your formulas for T_1 and T_∞ from questions 4a and 4b and then simplify your formula.
Brent's Lemma: $T_p \geq \max(T_1/p, T_\infty)$. With $T_1 = n^2$ and $T_\infty = 2n$, we get $T_p \geq \max(n^2/p, 2n)$. Therefore, speedup $S(n, p) = T_1/T_p \leq n^2/\max(n^2/p, 2n) = \min(p, n^2/(2n)) = \min(p, n/2)$.

(d) (**5 points**)
Derive the actual value of $SpeedUp(n, p)$?. In other words, what is the minimum number of time-steps to perform dynamic programming with p processors on inputs of length n assuming zero communication costs. Show that the optimal speed-up is greater than the lower bound from Brent's lemma, and give a one or two sentence explanation of why this is the case for this particular algorithm.
Feel free to simplify your formulas by ignoring $+1$ and -1 additive constants. Clearly state when you make such approximations – using a $\approx$ instead of $=$ is one qay to "clearly state...".
With p processors and diagonal parallelism, time per diagonal is $\lceil \text{diagonal\_size}/p \rceil$. The optimal parallel time $T_p \approx \max(n^2/p, 2n)$, giving speedup $S(n, p) \approx n^2/\max(n^2/p, 2n) = \min(p, n/2)$. This matches Brent's bound for this algorithm because the diagonal structure allows perfect load balancing.

(e) (**5 points**)
Show that $SpeedUp(n, \infty)$ can be achieved with fewer than ∞ processors. Give a formula the smallest number of processors that can be used to achieve $SpeedUp(n, \infty)$ as a function of n. We will write $p_{\max}(n)$ to denote this number of processors. Thus, $SpeedUp(n, p_{\max}(n)) = SpeedUp(n, \infty)$.
$S(n, \infty) = n/2$ from the span bound. This is achieved when $T_p = T_\infty = 2n$. Since the largest diagonal has n cells, once we have $p \geq n$ processors, we can process any diagonal in one step. Therefore, $p_{\max}(n) = n$.

(f) (**5 points**)
Let $E(n)$ denote the parallel efficiency when $p = p_{\max}(n)$. What is $\lim_{n\to\infty} E(n)$. Explain why the efficiency you calculated above is less than one. Refer to the causes of performance loss that we described in class.
$E(n) = S(n, p_{\max}(n))/p_{\max}(n) = (n/2)/n = 1/2 = 50\%$. Therefore, $\lim_{n\to\infty} E(n) = 1/2$. The efficiency is less than one due to **load imbalance**: not all diagonals have n cells. Diagonals near the corners have few cells, leaving many processors idle. Only the middle diagonals fully utilize all n processors.